Designing Enterprise Retrieval-Augmented Generation Systems

Technical Research Report

Name: Maryam Fatima
Track: NLP and AI Agents

Retrieval-Augmented Generation, or RAG, is a way of building AI systems that answer questions using an organization's own documents instead of relying only on what the AI model already knows. Instead of guessing, the AI first looks up relevant information and then writes an answer based on it. This makes answers more accurate and lets them be traced back to a real source. This report explains the main parts of a RAG system and how to design them well for enterprise use.

1. Architecture

A RAG system has two pipelines that work together. The first runs once, when a document is uploaded. The second runs every time a user asks a question.

- **Ingestion:** A file is uploaded, its text is pulled out, and cleaned up.
- **Query:** A question comes in, the system finds matching document chunks, and an AI model writes the answer using them.

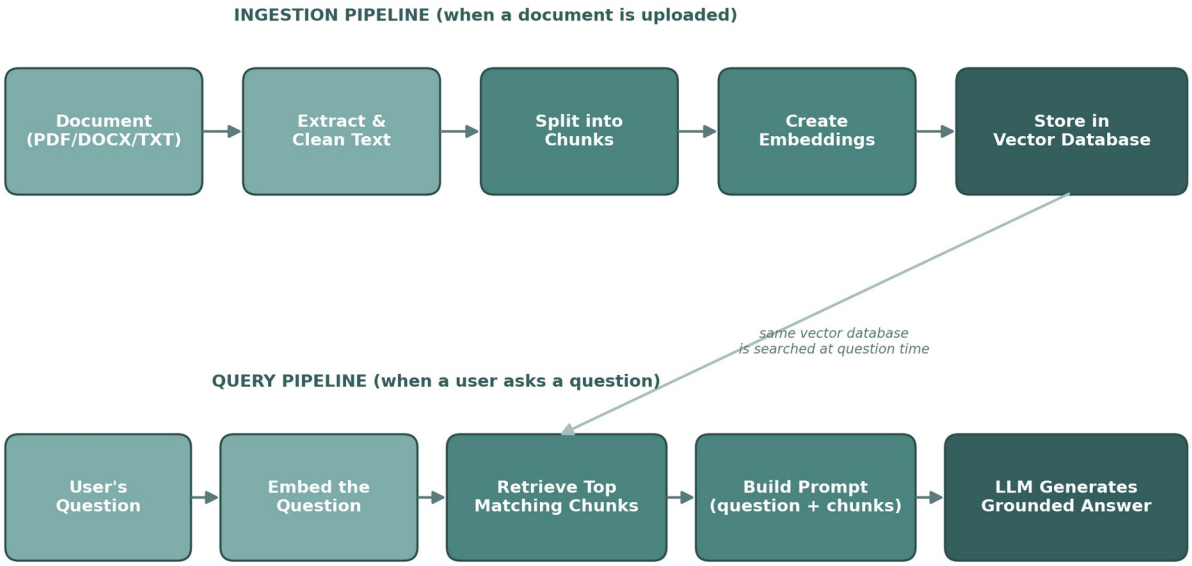

Figure 1: The two pipelines in a RAG system — ingestion (top) and query (bottom) — share the same vector database.

2. Embeddings

An embedding is a list of numbers that represents the meaning of a piece of text. Texts with similar meaning end up with similar numbers, even if they use different words. This is what lets the system search by meaning instead of by exact keyword match.

- **Documents:** Each chunk of a document is turned into an embedding once, when it is uploaded.

- **Questions:** The user's question is turned into an embedding the same way, so it can be compared fairly.
- **Common choices:** Google's gemini-embedding-001, OpenAI's text-embedding-3, and open-source models like BGE or E5.

For enterprise use, the same embedding model must be used for both documents and questions. Mixing models produces poor results, because each model's numbers mean something slightly different.

3. Chunking

Documents are too long to embed as one piece, so they are split into smaller chunks first. How this splitting is done has a big effect on answer quality.

- **Size trade-off:** Chunks that are too small lose context; chunks that are too large mix multiple topics together, making retrieval less precise.
- **Fixed-size with overlap:** Splitting a fixed number of characters at a time (for example, 1000 characters), letting the next chunk start slightly before the last one ended.
- **Semantic chunking:** Splitting by meaning — at paragraph or section breaks — so each chunk stays on one topic. More accurate, but more complex to build.

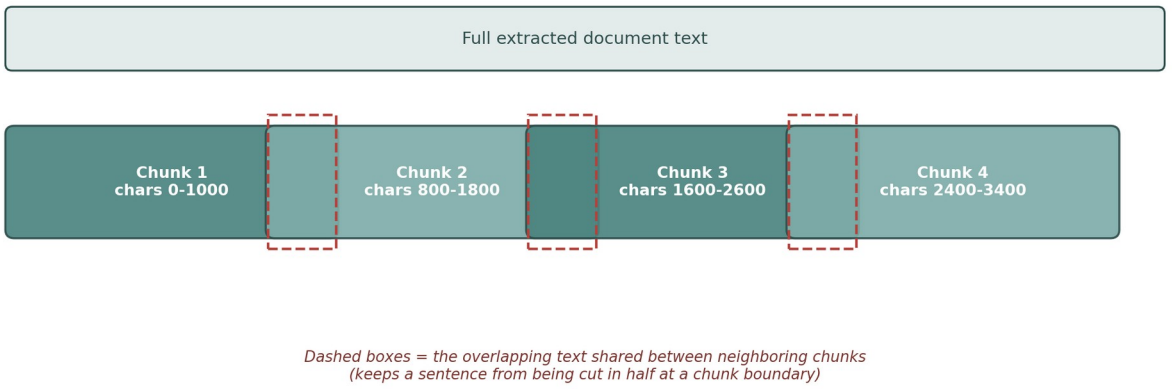

Figure 2: Overlapping chunks (dashed outlines) keep a sentence from being cut in half at a chunk boundary.

4. Retrieval

Retrieval is the step where the system finds which chunks are most relevant to a question.

- **Similarity search:** The question's embedding is compared to every stored chunk embedding, usually with cosine similarity or Euclidean distance.
- **Top-k:** Only the best-matching chunks (for example, the top 5) are kept and passed forward — not the whole document collection.
- **Hybrid search:** Combining meaning-based search with traditional keyword search often catches things pure vector search misses, such as exact product codes or names.
- **Re-ranking:** A second, more careful model re-scores the initial results to push the truly best matches to the top. Improves quality at some extra cost.

5. Prompt Construction

The prompt is the actual text sent to the AI model. A well-built prompt is what keeps answers grounded and on-topic.

- **Clear instructions:** Tell the model to answer only using the provided chunks, and to say so clearly if the answer isn't in them.
- **Labelled context:** Label each chunk with its source document, so the model can mention where information came from.
- **Conversation history:** Add the last few exchanges of the conversation so follow-up questions like "what about the second one?" make sense.
- **Question placement:** Put the question last, right before asking for the answer — this consistently improves accuracy with most models.

6. Vector Databases

A vector database stores embeddings and searches through them quickly, even when there are millions of chunks. Enterprise teams should choose one based on scale, hosting preference, and whether they need extra search features.

Database	Type	Best for
ChromaDB	Self-hosted, local	Small/medium projects, prototypes, full control
Pinecone	Managed cloud	Production apps that don't want to manage servers
Weaviate	Self-hosted or managed	Needs hybrid search and metadata filtering built in
FAISS	Self-hosted library	Very large scale, maximum speed, more setup work
Milvus	Self-hosted or managed	Large enterprise deployments needing high availability

Key things to check before choosing one: does it persist data safely, can it filter by metadata (like document owner or date), and can it scale as the number of documents grows.

7. Hallucination Reduction

Hallucination is when an AI states something confidently that isn't actually true. RAG reduces this, but doesn't remove it completely. These practices help further:

- **Strict grounding:** Instruct the model to use only the retrieved chunks, and explicitly allow it to say "I don't have enough information."
- **Source citations:** Show which document and chunk backed each answer, so a person can verify it themselves.
- **Lower temperature:** A lower temperature setting makes the model stick closer to the source text instead of getting creative.
- **Evaluation testing:** Regularly test the system with known questions and known correct answers to catch drift over time.

- **Human review:** For high-stakes answers (legal, medical, financial), keep a human in the loop before the answer is acted on.

8. Future Improvements

RAG systems are still evolving quickly. Some directions worth watching:

- **Hybrid search everywhere:** Combining keyword and vector search by default, rather than choosing one.
- **Smarter chunking:** Splitting documents by their actual structure and meaning, not just character count.
- **Multi-modal retrieval:** Letting the system search text, tables, and images together, instead of text only.
- **Agentic, multi-step retrieval:** Systems that decide on their own to run more than one search before answering a complex question.
- **Continuous evaluation:** Automatically tracking answer quality over time and flagging when it drops.

Conclusion

A well-designed enterprise RAG system depends on getting each stage right — clean chunking, matching embeddings, precise retrieval, disciplined prompts, a suitable vector database, and active hallucination controls. None of these stages work well in isolation; the overall answer quality is only as strong as the weakest stage in the pipeline.